Hogeschool van Amsterdam

AI Poker Bot:

Enemy Design, Probabilistic Decision-Making and Evolution in Heads-up Texas Hold'em

Hogeschool van Amsterdam

Alexander Bonnee, Reggie Schildmeijer

Abstract

We study simple poker agents in heads-up Texas Hold'em using a two-round model with a cap of three raises per hand. We compare a random baseline (v0), a hand-strength rule bot (v1), and a Monte Carlo equity bot (v2). To maintain fairness in the results, we employ paired seeds with seat swaps, fixed blinds and stacks. We report the win rate, profit per hand, stack difference, action frequencies, and hands per second, along with Wilson intervals.

Across 10 seeds and 2,000,000 hands per matchup, v1 beats v0 by 99.9%-win rate [99.6, 100.0], and v2 beats v0 by 100.0% [99.8, 100.0]. v2 also beats v1 by 93.3% [92.2, 94.4]. Mirror tests (v0 vs v0, v1 vs v1, v2 vs v2) centre near 50%, which supports that the protocol is unbiased. We observe that adding more betting chances hurts the random bot because it makes more coin-flip choices in growing pots, while structured bots press when ahead and save chips when behind.

A preliminary MCCFR bot (v3) did not perform well in this setting due to its limited training and lack of card abstraction. The work demonstrates that clear, low-cost methods can be tested fairly and that equity and price-based approaches yield a substantial and measurable gain. We outline the next steps toward full heads-up no-limit rules and staged game-theoretic training.

José Maria Martins

Index

1. Introduction

Poker has long been seen as one of the most challenging environments for artificial intelligence research. Unlike chess or Go, players must act under conditions of hidden information, reason about their opponents' beliefs and intentions, and manage risk and deception.

Among various poker variants, Texas Hold'em has been the primary focus of academic and industry research. Heads-up Hold'em (1 vs 1) is computationally easier than multiplayer formats while still maintaining the core complexity of poker. For these reasons, it has become a standard setting for initial research projects.

Recent landmark systems [1], [2] have matched or surpassed professional players; however, they typically require substantial computing power and intricate engineering. Our goal is different: we study simple yet principled agents under a shallow two-round abstraction with a 3-action betting interface (fold/call/raise; with a three-raise limit). We compare four bots:

- **v0 – Random**, a baseline for implementation sanity.
- **v1 – Rule-based**, hand-strength threshold with a small bluff rate.
- **v2 – Monte-Carlo equity**, rollout-based decision making.
- **v3 – MCCFR** [3], game-theoretic learning under our abstraction.

This R&D approach tackles the problem from a practical perspective, aiming to design and analyse a poker bot that balances enemy modelling with probabilistic decision-making. The project does not seek to reproduce the complexity of state-of-the-art poker AI, but rather to investigate a set of focused questions:

- How can probabilistic models be used to guide betting decisions under uncertainty?
- How can we ensure a neutral evaluation setup?
- What concrete gains do we get by moving from rule-based play (v1) to Monte-Carlo equity rollouts (v2)?

By exploring these questions, the project contributes to understanding how AI systems can operate in uncertain and adversarial environments, where optimal solutions are less critical than robust, adaptive strategies. Beyond poker itself, insights from this work extend to broader domains where decision-making under uncertainty and adversarial conditions are central, such as finance, cybersecurity, and negotiation systems.

2. Background & Related Work

Heads-Up Texas Hold'em (1v1) balances manageability with imperfect information: agents must act with hidden information, handle risk, and adapt to an opponent, yet the state space remains small enough for controlled experiments. In this project, we use a simple two-round abstraction with three actions (fold/call/raise) and up to three total raises per hand, which keeps the environment straightforward for clear comparisons while maintaining key strategic elements.

In this work, *enemy design* is treated as behavioural profiling grounded in what we log. Design choices surface in observable behaviours—specifically the overall action frequencies (shares of fold/call/raise across hands) and their relationship to outcomes. We relate these signatures to win-rate, profit per hand/stack difference, total hands, and throughput (hands/sec), all computed from per-hand logs and aggregate counters.

Modern poker AIs (e.g., Libratus; Pluribus) achieve professional strength via large abstractions and regret-minimisation with heavy compute. Such approaches are outside the scope of a short, single-author project. We instead study lightweight agents under a neutral, variance-controlled evaluation (seat-swaps, multi-seed runs) and report win-rate, profit per hand/stack difference, and action-frequency summaries.

Position of this work.

1. Probabilistic decision-making. Use hand-strength/equity signals and simple stochastic action mixing to guide betting choices in the two-round abstraction with three actions and up to three raises per hand.
2. Balanced evaluation. Enforce neutrality with seat-swapped, multi-seed matches; report win-rate, profit per hand/stack difference, total hands, hands/sec; and verify that random vs random lands near 50/50 with ≈zero profit per hand.
3. Evolution from rules to rollouts. Run thousands of automated hands to track whether a "good bot" consistently outperforms a random baseline and quantify the v1 → v2 upgrade in both outcomes (win-rate, profit per hand/stack difference) and behaviour (overall fold/call/raise frequencies).

3. Methodology

3.1 Environment setup

Experiments run on Windows 10 with Python 3.13 and a compiled C++ backend exposed via pybind11. The C++ engine is built with MSVC in Release mode using /O2 optimisation. Runs are driven by a consistent command-line interface (CLI) that supports flags for matches, hands per match, seeds, blinds, and stack sizes. The CLI emits timestamped CSV and JSON files that include both configuration and results. Build scripts and dependency specifications are provided in the repository artefacts.

3.2 Game Model

We model heads-up No-Limit Texas Hold'em (1v1) with a two-round abstraction:

- Preflop: blinds posted; SB (button) acts first.
- Postflop: all five community cards are revealed at once; BB acts first.

Legal actions are fold, check/call, and bet/raise. Actions alternate until both players check/call, someone folds, or the global cap of up to three raises per hand is reached. Bet sizing is supplied by the agent; the engine validates stack/pot legality and resolves showdowns with a bitset-based C++ evaluator. This abstraction preserves key trade-offs (position, pressure, value vs. bluff) while enabling very high throughput. We intentionally avoid emulating all four betting streets of HUNL to keep experiments controlled and fast.

Default setup: blinds 10/20, stacks 1000 chips, button alternates every hand, global cap of 3 raises per hand, and discrete bet sizes chosen by the agent.

3.3 Hand evaluation

Hand strength evaluation is implemented directly in C++ for optimal performance and consistency. Each card is mapped to a single bit in a 64-bit mask, enabling the evaluator to identify rank patterns and flush/straight structures through constant-time bitwise operations and minimal precomputed tables. The evaluator outputs a monotonically increasing score: higher scores indicate stronger seven-card hands. In the Python layer, we normalise the raw score into a [0, 1] strength proxy, maintaining monotonicity and permitting simple threshold-based decision rules. The post-flop (five-card board) strength thus accurately reflects the actual made hand. In contrast, preflop decisions rely on a pragmatic heuristic due to the fewer than five cards available for evaluation. This preflop heuristic is based on the average rank of the two-hole cards (normalised) with a capped bonus for pairs; suitedness and connectivity are deliberately excluded in V1 to maintain the base policy's interpretability and stability.

3.4 Evaluation Protocol

3.4.1 Metrics

The choice of evaluation metrics in poker AI research requires careful consideration of both statistical validity and practical interpretability. Unlike deterministic games where outcomes are purely strategic, poker involves significant stochastic elements that demand robust measurement approaches.

The match win rate serves as our primary performance indicator, but we augment it with Wilson confidence intervals [4] rather than relying solely on simple percentages. This choice reflects the reality that poker outcomes follow binomial distributions, and Wilson intervals provide superior coverage properties, especially when dealing with extreme win rates or small sample sizes. The method accounts for the inherent uncertainty in stochastic games where even optimal play cannot guarantee victory.

Profit analysis offers economic insight that win rates alone cannot provide. By tracking both absolute profit per match and normalised profit per hand, we can distinguish between bots that achieve high win rates through lucky variance versus those that demonstrate consistent strategic advantage. This economic perspective is crucial for understanding whether performance improvements translate to meaningful value extraction.

Action frequency analysis reveals the underlying decision-making patterns that drive performance differences. Rather than treating poker as a black box, we measure fold, call, and raise rates to understand strategic style. This granular view helps identify whether performance gains come from selective aggression, disciplined folding, or appropriate value betting.

Hand type breakdown tracks showdown performance across different poker hand categories, providing insight into which types of hands drive strategic advantages. This analysis helps validate that performance improvements stem from the selection of appropriate hands rather than fortunate variance in specific hand types.

3.4.2 Randomness & Fairness

The fundamental challenge in poker AI evaluation arises from the inherent randomness of the game, which leads to a complex interplay between strategic decision-making and stochastic elements in outcomes. Unlike chess or Go, where outcomes are primarily determined by player skill, poker involves random card dealing, random opponent actions, and random tie-breaking decisions that can significantly impact the results.

The randomness problem in computers begins with the fact that true randomness is impossible to generate algorithmically. Computer random number generators are pseudo-random, producing sequences that appear random but are completely deterministic given a starting seed. This creates a critical challenge: how do we ensure that our experimental results reflect genuine strategic differences rather than artefacts of our random number generation?

Seed dependency represents a particularly insidious problem in poker AI research. A single experimental run with a fixed seed may show one bot dramatically outperforming another, while a different seed may yield the opposite result. This variability can lead to false conclusions about strategic effectiveness if not properly managed. The problem is compounded by the fact that different random sequences can create systematically different game scenarios; some seeds might produce more favourable card distributions for certain playing styles.

Our multi-seed solution addresses these challenges through systematic seed isolation. We use a global seed to control deck shuffling and match ordering, ensuring reproducible game scenarios across runs. Each bot receives an independent seed with large offsets (spacing of 1000 or more) to prevent correlation between their random decisions. This approach ensures that different experimental runs represent truly independent samples while maintaining full reproducibility.

For this paper, we use 10 seeds, 200 matches, and 200 hands per match.

3.4.3 Confidence Intervals

Statistical analysis in poker AI evaluation requires careful consideration of the underlying probability distributions. Traditional standard approximation methods become unreliable when dealing with extreme win rates or small sample sizes, which are common in poker research.

Wilson score intervals provide superior accuracy for binomial proportions, especially when dealing with win rates near 0% or 100%. The method accounts for the discrete nature of win/loss outcomes and provides better coverage properties than the normal approximation. This is particularly important in poker AI research, where we often encounter extreme performance differences between sophisticated and naive strategies.

3.4.4 Throughput & Compute Budget

We include hands/sec to make compute trade-offs explicit: rollout-based agents are slower than rule-based ones. It helps one analyse the trade-offs between performance and speed.

3.5 Limitations

The betting system abstraction is the main methodological limitation. The simplified two-round model (preflop and postflop with all five board cards revealed at once) omits several strategic layers present in full heads-up No-Limit Hold'em. These missing elements include intermediate betting rounds (flop and turn), multi-street pressure buildup, and the exploitation of evolving board textures. Moreover, the three-raise cap per hand artificially limits late-stage aggression and complex betting sequences that would normally develop in an uncapped game.

This abstraction emphasises computational efficiency and interpretability over complete game-theoretic realism, allowing for rapid simulation and precise strategic analysis at the expense of strategic depth. The trade-off is justified by the study's aim to compare algorithmic approaches rather than to achieve optimal play, as shown by the performance hierarchy established between bot variants.

4. Bots – Design and Decision Policies

4.1 RandomBot (v0)

RandomBot serves as our statistical baseline, implementing a completely non-strategic approach that makes decisions using uniform random selection across all available actions. This bot serves as our null hypothesis: any performance differences observed in other implementations must reflect genuine strategic advantages rather than implementation artefacts or systematic biases in our simulation framework. The RandomBot's decision-making process is intentionally simplistic but carefully designed for statistical validity. When facing a bet, the bot randomly selects from all legal actions (fold, call, raise) with uniform probability. When no bet is present, it randomly chooses between checking and betting with equal probability. This uniform random selection ensures the bot cannot make strategically optimal choices regardless of hand strength or game context.

Technical Implementation: The bot uses three fixed bet sizes (50%, 75%, or 100% of the current pot) selected uniformly at random, providing sufficient variance in betting behaviour while remaining computationally trivial. Each bot instance maintains its own random number generator object with an independent seed, ensuring that different bot instances produce uncorrelated behaviour.

4.2 HandStrengthBot (v1 — rule-based)

V1 is our first step beyond randomness: a strength-driven policy where stronger hands play more aggressively and weaker hands back off. The design is deliberately simple and transparent, allowing us to isolate the effect of basic strength signals.

Signals

> **Preflop.** A heuristic maps the average rank of the two-hole cards to $[0,1]$, with a capped pair bonus (e.g., +0.3) to reflect pair value in heads-up. The preflop heatmap displays the resulting strength surface and the decision bands we use (fold < 0.15, call 0.15–0.30, and raise > 0.30).

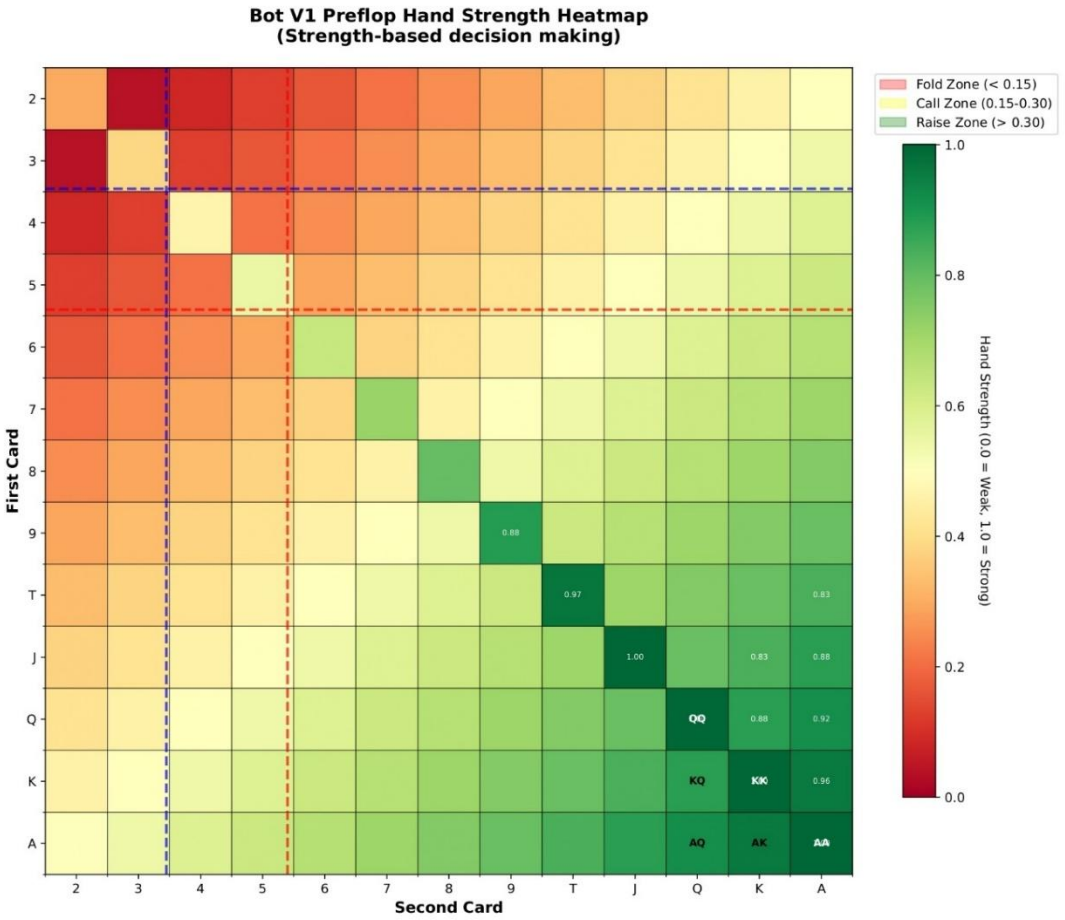

Figure 4.1 — Preflop strength heatmap (v1).

Postflop. We switch to the C++ evaluator (seven-card, bitset/LUT) and normalise to $[0,1]$. The hand-class chart illustrates typical postflop strengths (e.g., pairs ≈0.47, two pair ≈0.64, etc.) and overlays our decision thresholds.

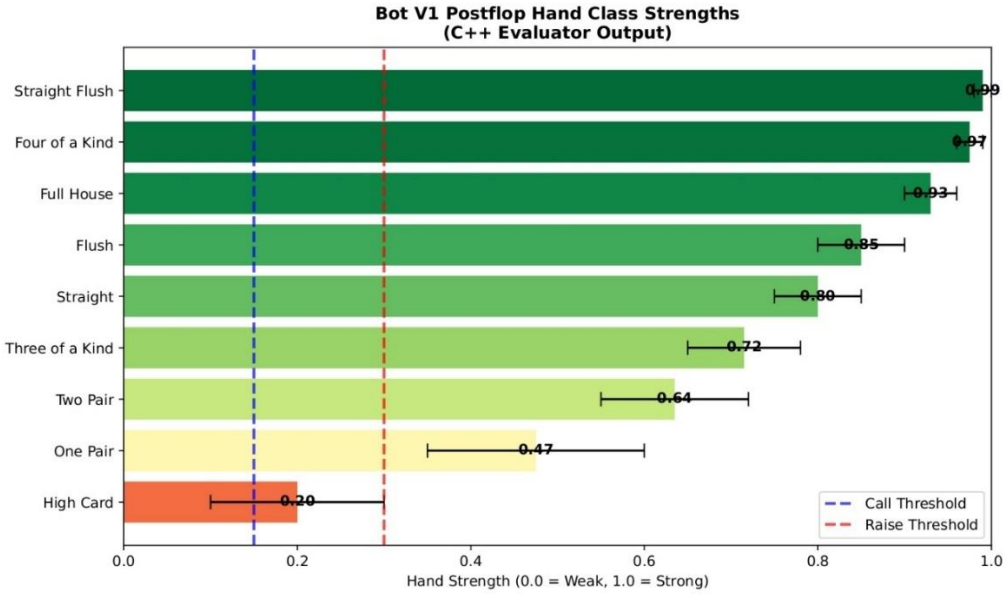

Figure 4.2 — Postflop hand-class strengths (v1)

Decision policy

When checking is allowed, the bot bets/raises if its strength is ≥ 0.30; otherwise, it checks, with a minor bluff frequency of 0.05 to avoid being completely face-up. When facing a price, call if the strength is ≥ 0.15; otherwise, folds. On any bet/raise, value or bluff, it samples the size uniformly from {0.5, 0.75, 1.0, 1.5} × pot, and the engine enforces legality and the three-raises-per-hand cap.

Defaults: raise_threshold = 0.30, call_threshold = 0.15, bluff_frequency = 0.05, bet_size = {0.5,0.75,1.0,1.5} × pot.

Scope & limits. v1 does not incorporate position adjustments, opponent modelling, pot-odds/range calculations, or street-specific heuristics; this is intentional to keep the baseline interpretable and reproducible. It gives us a clean rung above v0 to measure the gains from probabilistic rollouts (v2).

4.3 MonteCarloBot (v2 – rollout-based)

v2 moves beyond fixed thresholds to an equity-driven, context-aware policy. Instead of asking "is my hand strong enough for a preset band?", v2 estimates win equity via Monte-Carlo rollouts and compares it to the price being offered, then nudges that comparison with small, interpretable adjustments for stack state, board texture, position, and a coarse opponent profile.

Equity and price. Preflop, we keep v1's efficient heuristic (average rank with a capped pair bonus) to save compute. Postflop, we run rollouts (default 300 simulations) against random opponent holdings and compute.

$$equity = \frac{(wins + 0.5 \times ties)}{simulations}$$

When facing a bet, the price to continue is

$$price = \frac{to_{call}}{(pot + to_{call})}$$

The core decision is then framed as follows: raise if equity comfortably exceeds the price; call if it modestly exceeds the price; otherwise, fold/check.

Context adjustments. We apply light, additive adjustments that keep behaviour interpretable. A stack-aware aggression term pushes the margin down when ahead and up when behind:

$$\boldsymbol{\Delta stack = k1 \times \left(\frac{(current_{stack} - starting_{stack})}{starting_{stack}}\right) - k2 \times \left(\frac{to_{call}}{\max(1, pot)}\right)}$$

with defaults $k_1 = 0.05$, $k_2 = 0.02$. Board texture shifts are minor and symmetric: on *wet* boards, we add about +0.02 to the call margin and +0.03 to the raise margin (more selective); on dry boards, we subtract roughly 0.01/0.02 (more assertive); paired boards get a slight caution bump. We offer a positional bonus of +0.02 equity when acting last, and a single tightness parameter (0 loose … 1 tight; default 0.5) adjusts margins in favour of tight profiles and against loose ones.

Decision rule (prose). Starting from raw equity, we apply the positional bonus and opponent/texture adjustments, then compare the adjusted equity to the price using two margins: raise_margin and call_margin (both small, e.g., 0.04 and 0.01 before adjustments). If adjusted equity ≥ price + adjusted raise_margin, we bet/raise; else if adjusted equity ≥ price + adjusted call_margin, we call/check; otherwise we fold/check. This produces aggressive value when well ahead of price, disciplined calls when barely ahead, and folds elsewhere.

Semi-bluffing and draws. When a flush/straight (or combo) draw is detected, we allow semi-bluffs with probability.

$$p(semi_{bluff}) = semi_{bluff_{freq}} \times draw_{strength}$$

where defaults: semi_bluff_freq 0.25, draw_strength in $[0,1]$); we still require baseline equity around 30% so bluffs are not gratuitous.

Sizing. Bet/raise size is equity-tiered: ≳80% equity → about 1.0× pot; 60–80% → 0.75× pot; 40–60% → 0.5× pot; <40% → 0.25× pot (for bluffs/semi-bluffs). Sizes are then clamped by the current pot, the three raises per hand cap, and at most 80% of the remaining stack to avoid pathological all-ins in our abstraction.

Defaults (for reproducibility). Rollouts = 300; raise_margin = 0.08; call_margin = 0.05; $k_1 = 0.05, k_2 = 0.02$; semi_bluff_freq = 0.25; positional bonus = +0.02; tightness = 0.5.

Performance note & scope. V2 is slower than v1 due to rollouts; we mitigate this with preflop heuristics and simple sizing tiers, which kept throughput acceptable for our evaluation plan. Within the two-round abstraction, we do not perform multi-street planning or rich opponent exploitation; those are left for future work.

4.4 MCCFR (v3) — Preliminary / Failure Analysis

We prototyped a Monte-Carlo Counterfactual Regret Minimisation (MCCFR) agent under the same two-round abstraction. In our setup, v3 did not achieve competitive performance versus v1 (rules) or v2 (rollouts), and we therefore exclude it from the main comparison tables.

Why did it struggle? The primary issues were (i) **insufficient training** relative to the size of the information set and (ii) **no card abstraction**. With exact cards and even a coarse action set, the state space is too large for the training budget we could allocate; regret updates remained sparse and unstable. Our two-round model (all five board cards revealed at once) also reduces the sequential structure CFR usually exploits, making counterfactual reach probabilities and value signals noisier in this setting.

The consequence was that the learned policy did not improve reliably and underperformed v1/v2 across seeds and seat-swaps. To avoid drawing weak conclusions about CFR itself, we treat v3 as preliminary and omit it from the headline results.

5. Results

Unless stated otherwise: 10 seeds, 200 matches/seed, 200 hands/match (→ 10,000 matches, 2,000,000 hands), seat-swaps per seed. We report win-rate (A) with 95% CIs, stack difference (in chips, zero-sum), hands/sec, and wall time.

Table 5.1 — Summary of matchups

Experiment	Win-rate A	95% CI	Stack difference	Hands/sec	Time
v1 (HandStrength) vs v0 (Random)	99.9%	[99.6, 100.0]	+1966	98,851	2.4s
v2 (MonteCarlo) vs v0 (Random)	100.0%	[99.8, 100.0]	+1999	1,529	108.7s
v2 (MonteCarlo) vs v1 (HandStrength)	93.3%	[92.2, 94.4]	+1620	969	257.3s
v2 (MonteCarlo) vs v2 (MonteCarlo)	48.8%	[46.6, 51.0]	-39	364	850.1s
v1 (HandStrength) vs v1 (HandStrength)	49.6%	[47.5, 51.8]	-8	29,463	10.7s
v0 (Random) vs v0 (Random)	49.9%	[47.7, 52.0]	-5	104,812	4.1s

5.1 Baseline sanity (v0 vs v0)

Random vs Random lands at **49.9%** (95% CI **[47.7, 52.0]**) with **≈ 0** stack drift. This confirms that shuffling, legality checks, seeding, and logging are neutral. The very high throughput indicates the simulator overheads are minimal in the absence of rollouts.

5.2 Rule-based vs random (v1 vs v0)

V1 all but eliminates chance as a differentiator: **99.9%** (95% CI **[99.6, 100.0]**), large positive stack swing, and near-instant throughput. Simple strength bands plus a tiny bluff rate are enough to punish v0's coin-flip calling/folding. Value hands grow the pot; weak hands avoid it, exactly what a baseline strategic policy should do.

5.3 Rollouts vs random (v2 vs v0)

V2 is essentially perfect against v0: **100.0%** (95% CI **[99.8, 100.0]**). The slower throughput reflects postflop rollouts, not instability. Equity-vs-price decisions prevent spew and harvest value wherever the price is favourable, so even random spikes don't translate into sustained wins for v0.

5.4 Rollouts vs rules (v2 vs v1)

V2 beats v1 decisively at **93.3%** (95% CI **[92.2, 94.4]**), with a significant positive stack difference. The upgrade comes from replacing static thresholds with equity-aware decisions: marginal v1 checks become thin value bets when the price allows; marginal v1 calls become disciplined folds when the price is unfavourable. Small context nudges (position, texture, stack state) help, but the equity-vs-price core does most of the work.

5.5 Mirror and position checks

Mirror matches centre on **50%** with CIs covering 50% (v1: **49.6%**, v2: **48.8%**). That's what we want under a paired-seed, seat-swapped protocol: no systematic seat advantage and no hidden bias in the simulator. The residual deviation (approximately 1%) is well within the sampling noise at these scales.

5.6 Throughput and cost

Speed follows policy complexity: **v0 > v1 > v2**. Relative to v1, v2 is substantially slower due to rollouts, and the v2-vs-v2 pairing is the slowest. Reporting hands/sec alongside results makes the trade-off explicit: rollouts buy accuracy at the cost of time, and the gap is predictable from design.

5.7 Why do more raise opportunities widen the gap

A non-strategic policy like v0 treats every decision as a coin-flip. Additional raise opportunities therefore add more ways to bloat pots with weak holdings and give up value with strong ones. Structured policies (v1/v2) do the opposite: press when ahead, conserve when behind, so increasing decision points widens the edge. Even within our two-round model and three-raises-per-hand cap, we already observe large margins; adding more betting rounds would, in expectation, further impair v0 relative to v1/v2.

5.8 Synthesis

Across 2M-hand evaluations per matchup, the evaluation looks neutral, the baselines behave as intended, and the hierarchy is clear: **v2 > v1 > v0**. V1 demonstrates how far simple, interpretable rules can go; v2 shows that equity-aware, price-sensitive play delivers a considerable, statistically precise improvement, at a transparent computational cost.

6. Future Work

From abstraction to full heads-up no-limit (HUNL). The next step is to replace the two-round abstraction with the full four-street game (preflop, flop, turn, river), remove the artificial three-raise cap (leaving only stack constraints of true no-limit), and allow multiple actions per street (including check-raises and re-raises). This will surface delayed aggression, equity realisation across streets, and value/bluff balance that our current model compresses. Adapting our agents means exposing street-aware signals (preflop vs. flop vs. turn vs. river), widening the sizing menu, and revalidating neutrality under seat-swaps. The evaluation protocol should include street-level logging, allowing us to report preflop participation and aggression metrics alongside win rate and profit/hand.

Reviving v3 (MCCFR) with the proper scaffolding. Rather than training MCCFR directly on full HUNL, we should stage it: first, verify convergence on tiny games (Kuhn, then Leduc) to validate code and hyperparameters; next, reintroduce Hold'em with card bucketing (e.g., equity- or histogram-based clustering) and a modest action abstraction, using external-sampling MCCFR with regret-matching. Training should checkpoint average strategies and track progress via head-to-head tests and simple exploitability proxies under larger iteration budgets. Once stable on the abstracted game, we can gradually relax the abstraction and compare it to v1/v2 in the full HUNL protocol.

Beyond heads-up: multi-player no-limit Hold'em. Extending to 3–6 players introduces non-trivial dynamics (multiway pots, non-zero-sum slices per hand, amplified position effects, and combinatorial action growth). The simulator should support rotating seats, per-seat reporting, and a round-robin cross-play evaluation (such as tournaments or league tables) rather than simple A/B matches. Neutrality checks must be generalised (e.g., mirror runs across seats), and results should be aggregated both per-seat and overall. This direction makes the project closer to "normal" poker while stress-testing scalability, protocol design, and the robustness of our agents in truly adversarial, multi-agent settings.

7. Conclusion

This project developed a small, repeatable setup for heads-up Texas Hold'em, featuring two rounds and a cap of three raises per hand. We ensured the evaluation was fair by using paired seeds and seat swaps. We recorded win rate, profit per hand, stack difference, action frequencies, and hands per second, and we used Wilson intervals to report uncertainty. Our goal was not to match the most advanced poker AIs, but to see which simple ideas really help.

Across two million hands per matchup, the ranking is clear: the strength-based bot (v1) beats the random bot (v0), and the equity-and-price bot (v2) clearly beats v1. Mirror tests (v1 vs v1, v2 vs v2) centre on 50%, which supports that the protocol is unbiased. We also see that more betting chances make the random bot worse, because it flips more coins in big pots while the structured bots press when ahead and save chips when behind.

Two lessons stand out. First, comparing equity to price, combined with neutral controls and simple behaviour summaries, gives a strong and reliable gain over fixed thresholds. Second, our MCCFR prototype (v3) did not work here because we lacked the needed training time and card abstraction. That does not mean CFR is bad; it means it needs the right setup. Within the limits of our model, we demonstrated that simple, clear agents can be tested fairly and that rollouts provide a substantial, measurable benefit. Next steps are to move toward full heads-up no-limit rules, add street-level logging, and bring back CFR with a staged plan.

References

[1] N. B. a. T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science,* vol. 359, nº 6374, pp. 418-424, 2018.

[2] N. B. a. T. Sandholm, “Superhuman AI for multiplayer poker,” *Science,* vol. 365, nº 6456, pp. 885-890, 2019.

[3] K. W. M. Z. a. M. B. M. Lanctot, “Monte Carlo Sampling for Regret Minimization in Extensive Games,” em *Advances in Neural Information Processing Systems (NeurIPS)*, Online, 2009.

[4] T. T. C. a. A. D. L. D. Brown, “Interval Estimation for a Binomial Proportion,” *Statistical Science,* vol. 16, nº 2, pp. 101-133, 2001.